Hochschule Konstanz
Fakultät Informatik

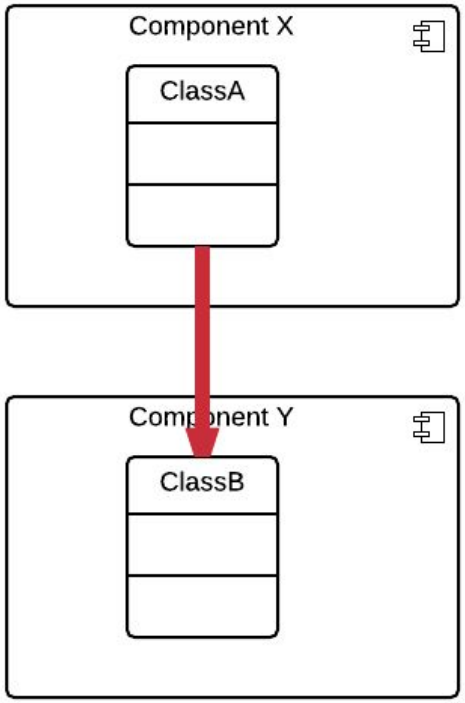
Software Engineering

11 - Dependency Injection

Prof. Dr. Marko Boger

Direct Dependency

Class A directly depends on Class B. Component Y cannot be exchanged without changing Class A/Component X.

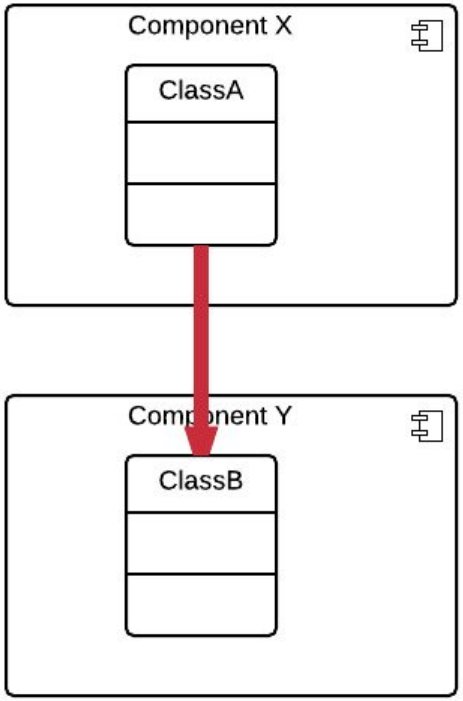


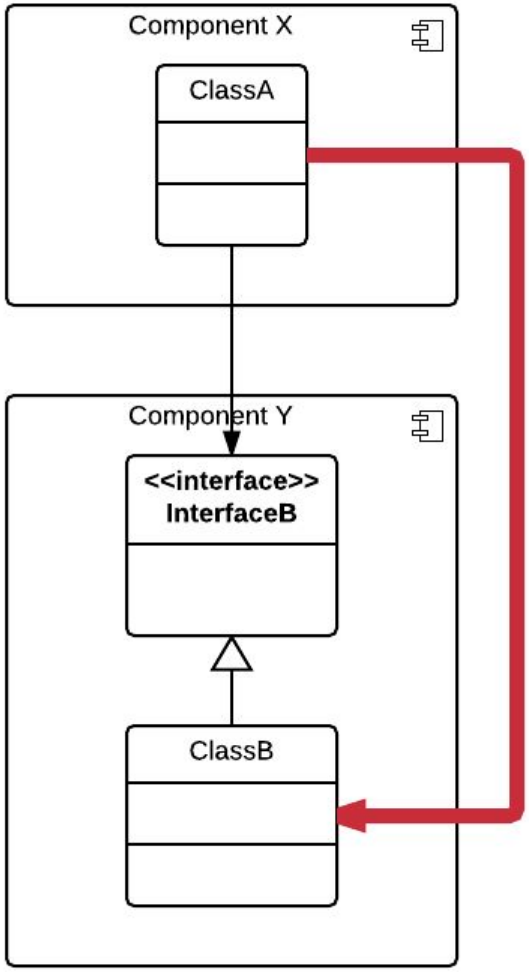
Dependency on Interface

Now Class A depends on the Interface of B. But to create an instance of B it still needs to call

`new ClassB`

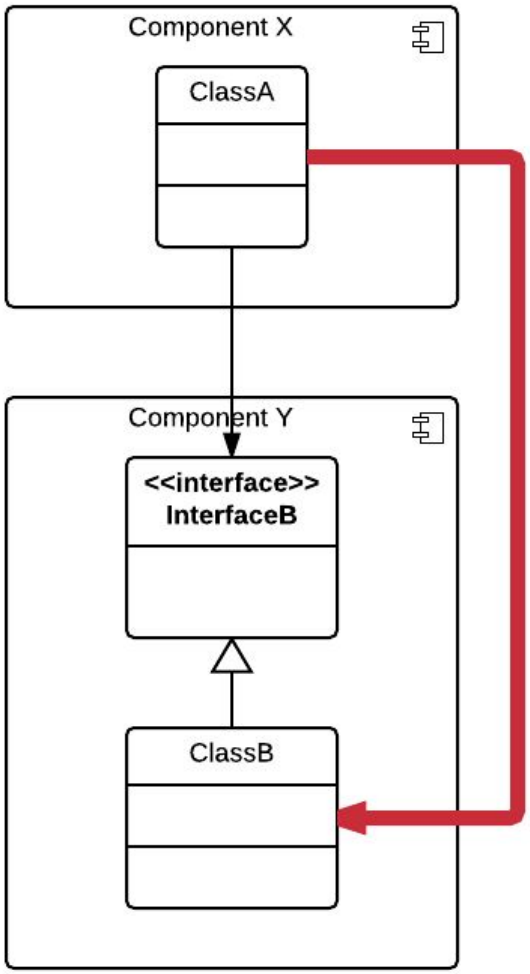
directly.

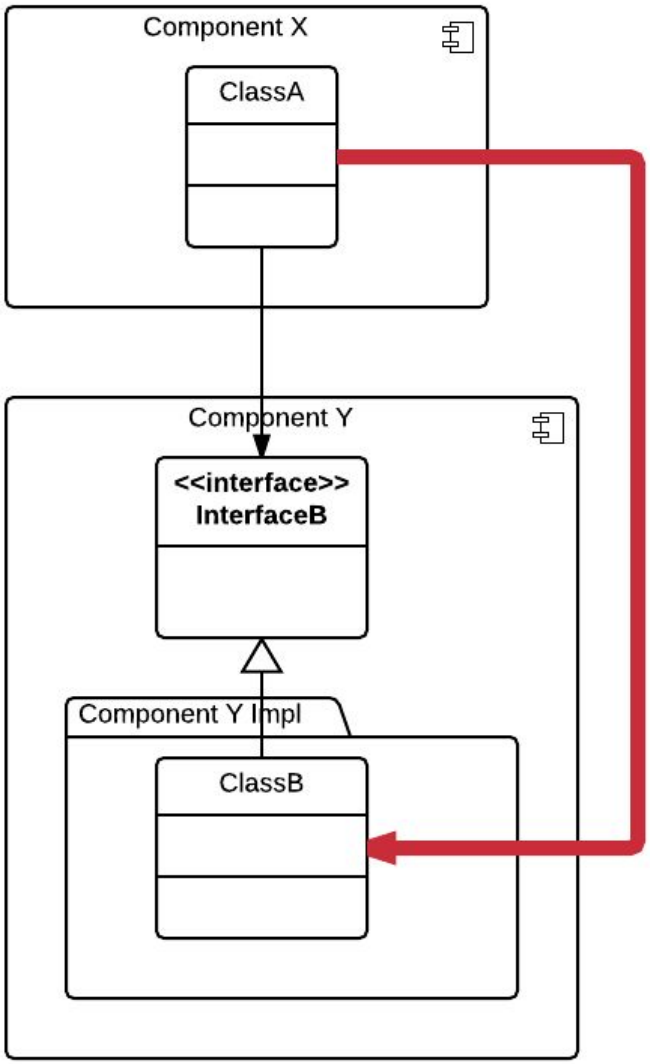




Implementation Package

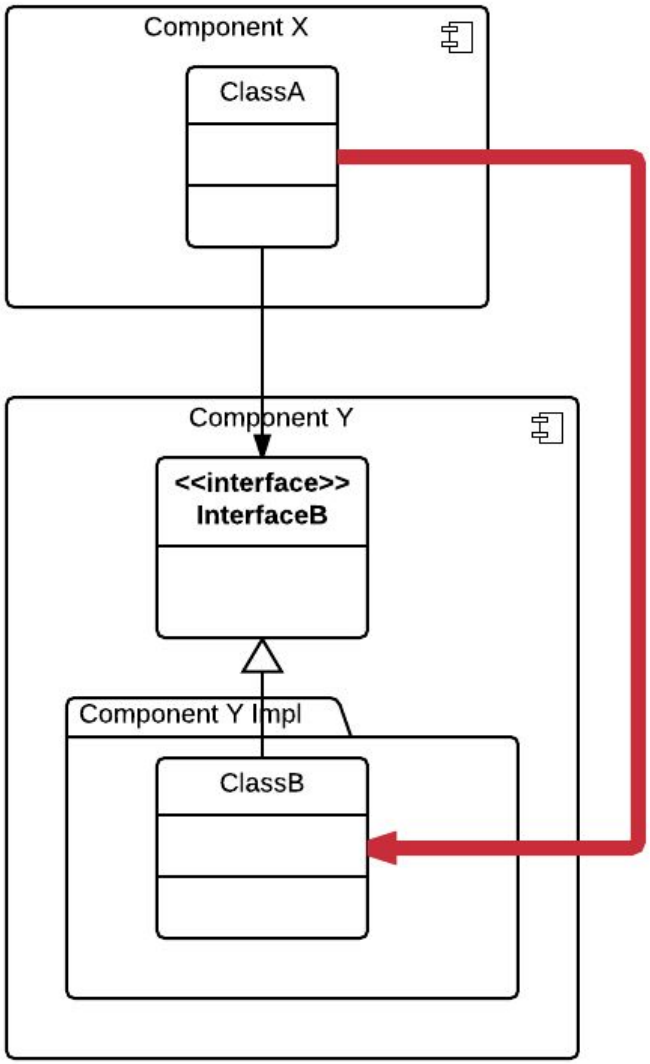
Now the implementation can be exchanged, but in Class A the import path still has to be changed.

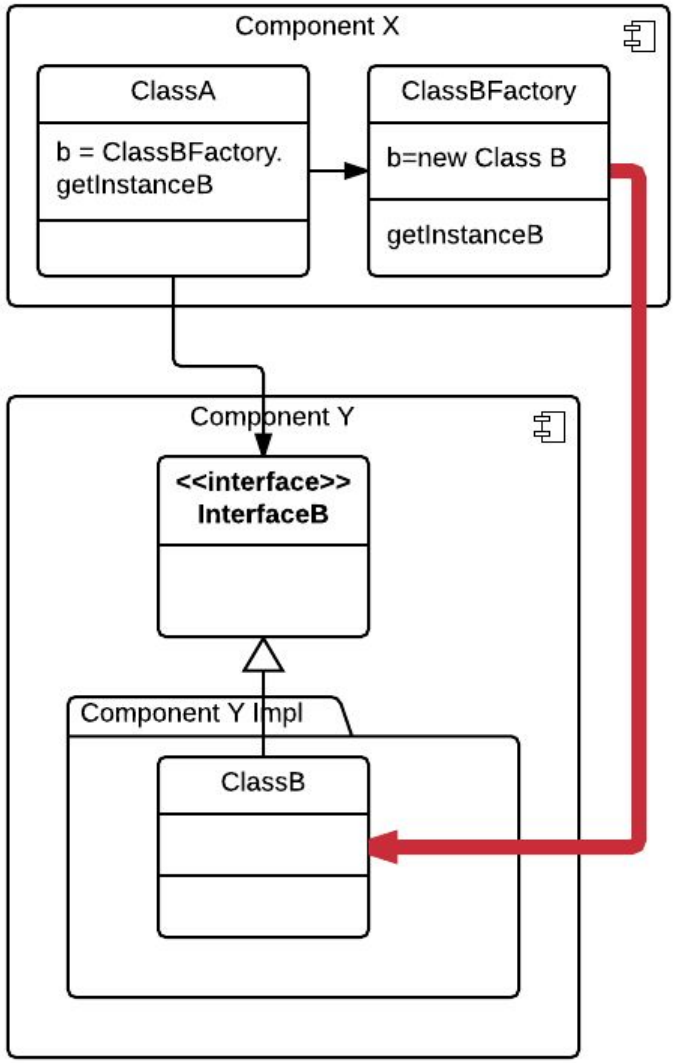




Factory

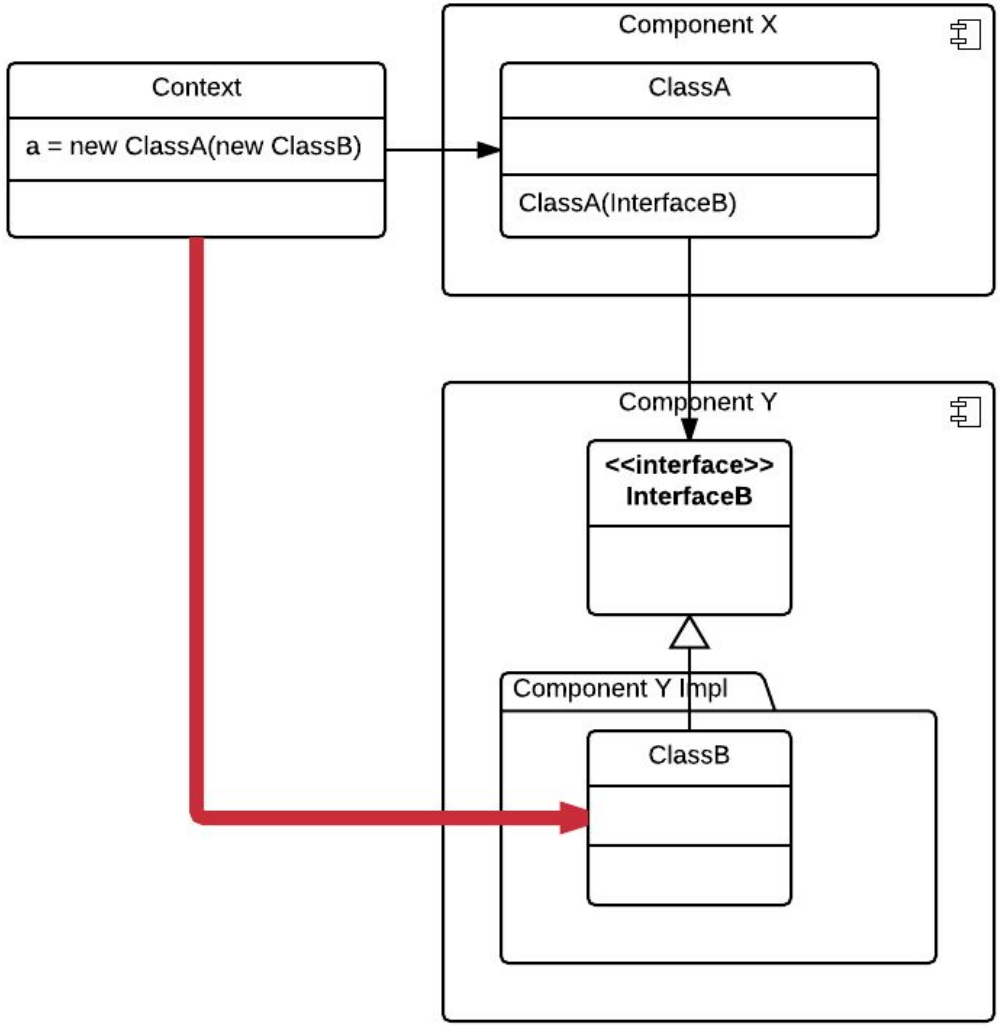
The dependency can be delegated to a Factory. This can be placed in Component X or Y.





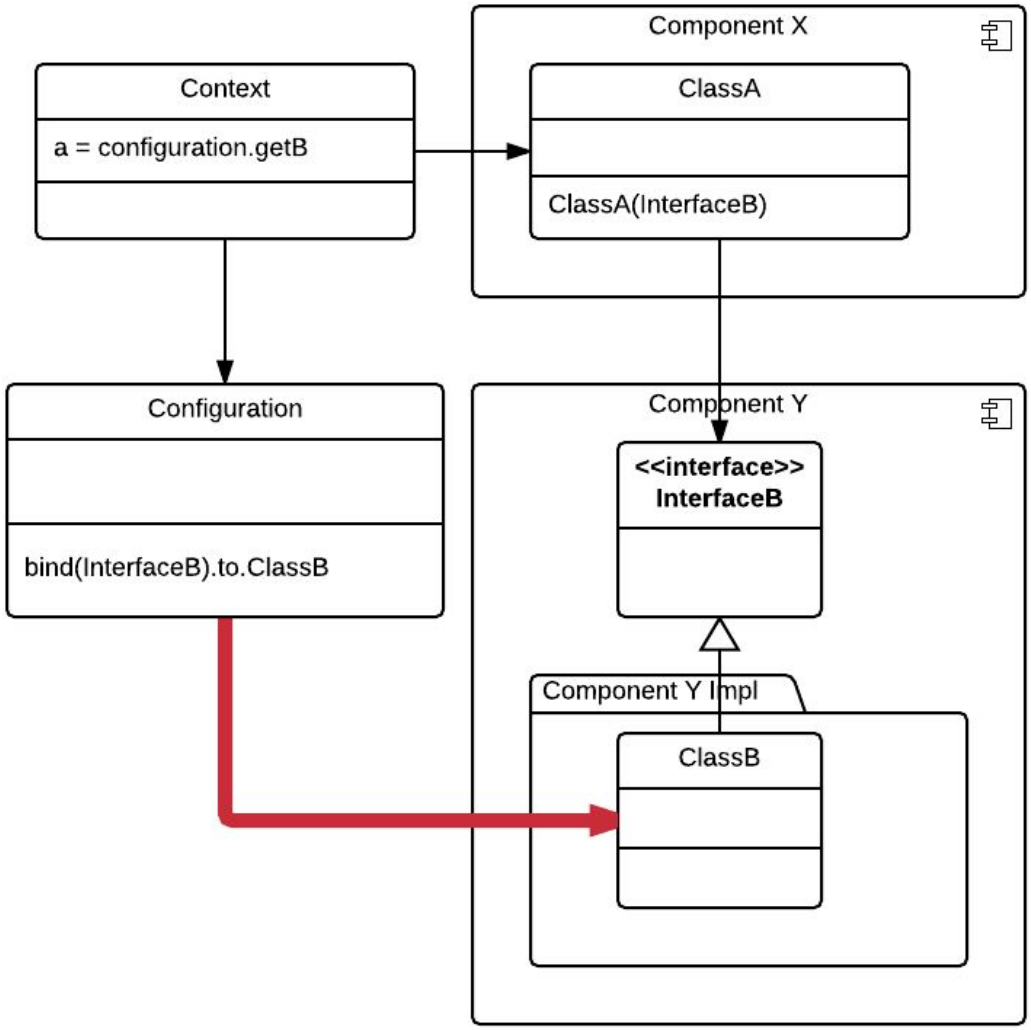
Dependencies in Constructors only

If all dependencies are parameters of the constructor and the creation is always done in a higher level Context the implementation can easily be exchanged in the Context. This is already a very clean situation.



Dependency Injection

If we can get the new instance from a high level configuration, there would only be one place in which all implementation exchanges would take place. This is a pattern called Dependency Injection. It works best if dependencies are only in the Constructors.



DI Frameworks

DI Frameworks can make use of the DI Pattern and make the introduction simple. They essentially work by providing Factories on the fly.

They need to know the dependency hierarchy from a configuration and often work with annotations.

Examples of DI Frameworks include

- Google Guice
- Spring DI
- Pico Container

Google Guice

Google Guice is a lightweight DI Framework.

- it is written in Java
- configuration is done in Java
- Compiler can find errors in the configuration

Scala Guice

Scala Guice is a wrapper for Google Guice that keeps the nature of it but adds Scala syntax.

Import it from sbt

```
libraryDependencies += "net.codingwell" %% "scala-guice" % "7.0.0"
"
```

Defining the Interfaces

```
import com.google.inject.{AbstractModule, Guice, Inject}
import net.codingwell.scalaguice.ScalaModule

trait Payment {
 def pay
}

class PaymentInCash extends Payment {
 override def pay = println("I am paying with cash")
}

class PaymentWithCard extends Payment {
 override def pay = println("Here is my credit card")
}

class Order @Inject() (payment: Payment){
 def handleOrder: Unit = {
   payment.pay
 }
}
```

Creating the Instance

```
class OrderModule extends AbstractModule with ScalaModule {
 def configure(): Unit = {
   bind[Payment].to[PaymentWithCard]
 }
}

import net.codingwell.scalaguice.InjectorExtensions._

object OrderServer {
 def main(args: Array[String]) {
   val injector = Guice.createInjector(new OrderModule())

   val orderService = injector.instance[Order]
   orderService.handleOrder
 }
}
```

Guice in Scala 3

For Scala 3 the syntax for Google Guice has changed a bit:

```
//Klasse mit Bindings
class WizardModule extends AbstractModule  {
 override def configure(): Unit = {
   bind(classOf[IController]).to(classOf[baseImplementation.Controller])
   bind(classOf[FileIO])
      .to(classOf[xmlIO.FileIOXML])
   bind(classOf[IPlayer])
     .to(classOf[Player])
 }
}

//Erstellung und verwendung des Injectors in einer Klasse
class Controller() extends IController {
  val injector = Guice.createInjector(new WizardModule)
  val fileIo = injector.getInstance(classOf[FileIO])
```

Given/Using for DI in Scala 3

A new mechanism in Scala 3 makes DI even simpler.

The implementations for interfaces can be “given” on module level

```
object Default {
  given  IController = Controller()
}
```

And then be used inside the module

```
import de.htwg.se.wizard.controller.modules.Default.{given}

class TUI(using controller: IController) extends UI {
```

Task: Use Dependency Injection

Define a Module
Bind your Interfaces of your Components to Instances here
Create an Injector in your main
Create Instances of your Components
Inject your Component Instances into the other Components